Datenbanken
VL-2, Historische Datenmodelle

3MI25-1

Dresden, 10/2026

Dozent: Dr. Harald Pfefferkorn

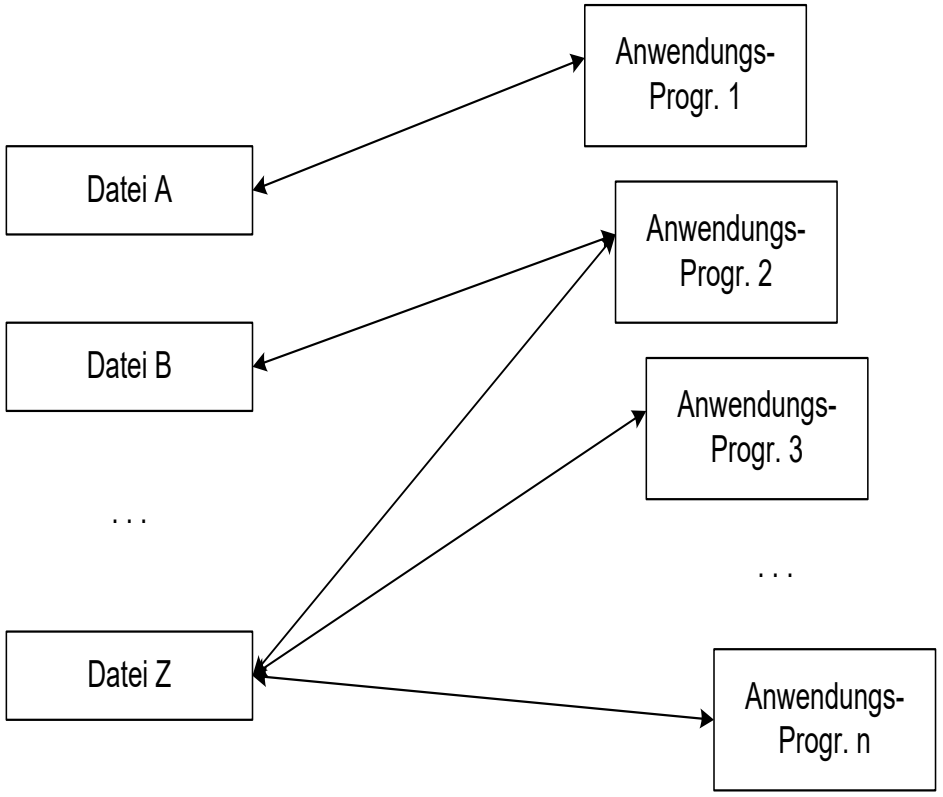
DHSN
DUALE HOCHSCHULE
SACHSEN

Historische Entwicklung

Hierarchisches Datenmodell

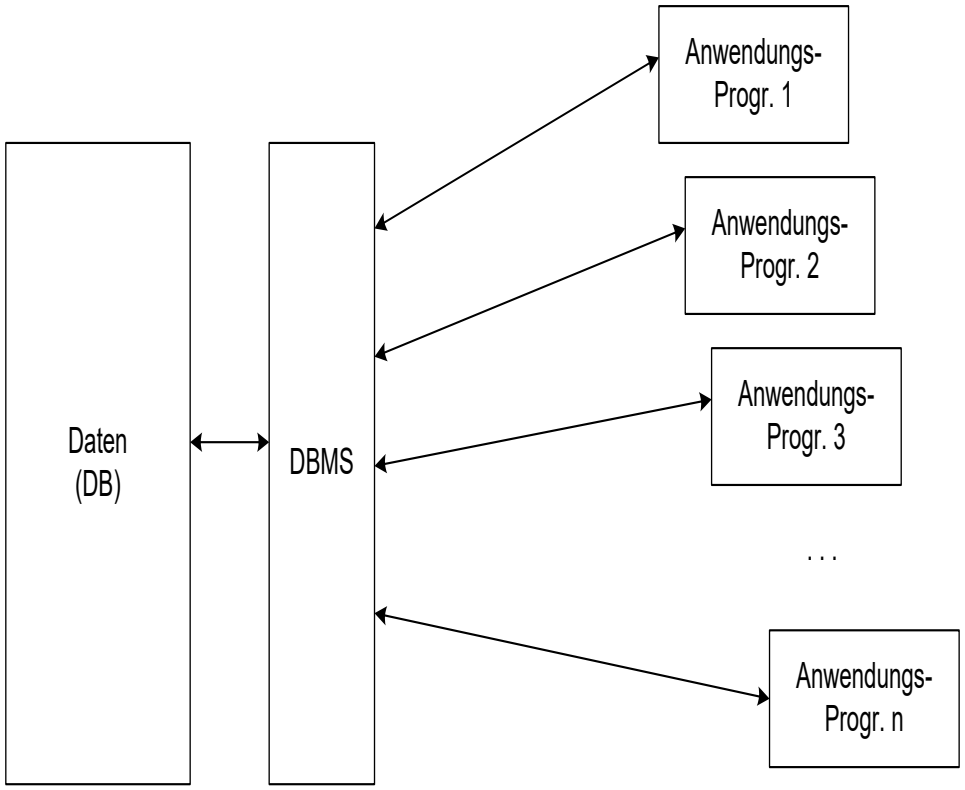
Netzwerkmodell

Arbeiten mit Dateien



- Datenstruktur (DS) ist im Programm definiert.
- Bei Änderung der Datenstruktur muss jedes Anwendungsprogramm angepasst werden
- Daten werden –in Abhängigkeit von den jeweiligen Anwendungsprogrammen- oft mehrfach abgespeichert (=Redundanz):
- Bei Änderungen sind Inkonsistenz möglich/wahrscheinlich
- Jeder Nutzer der Daten muss physische Struktur kennen.

Arbeit mit einem Datenbanksystem



- Anwendungsprogramme greifen über das DBMS auf die Daten zu
- DBMS ist Zugriffsfilter
- Identische Daten werden nur einfach abgespeichert
- Bei Änderung der Datenstruktur muss nur einmal im DBS geändert werden.

Vergleich Dateisysteme / DB-Systeme

	DB-System	Dateisystem
Abstraktion	logisch	physisch
Abspeicherung	In Datenbanken	In Dateien
Beschreibung der Daten	Zentrales Data Dictionary	Im Anwendungsprogramm
Zugriff	Über DB-System	Über Betriebssystem
Dienstprogramme (Backup, Recovery)	Über DB-System	Eigene Programmierung
Verknüpfung der Daten	Über DB-System	Eigene Programmierung

Beispiel (alle Kunden aus Darmstadt)

DB-System:

```
SELECT name, plz, ort
FROM   kunde
WHERE
          ort = ´Darmstadt‘,
```

Dateisystem (in C++):

```
#include <fstream.h>
..
typedef struct {
    char      name [30];
    int       plz;
    char      ort [30];
} kunde_record;
kunde_record buffer;
fstream kunden_file;
file.open (“kunden.dat“, ios::in | ios::binary);
while (!file.eof())
   {          file.read( (char*) &buffer, sizeof(kunde_record) );
    if buffer.ort = “Darmstadt“
         {     count << buffer.name << …
         }
    }
…
```

Historische Entwicklung von Datenbanksystemen

- 1965: Firma IBM entwickelt das DBS: IMS, das auf dem *Hierarchischen Datenmodell* beruht
- 1969 Standardisierungsgremium CODASYL schlägt *Netzwerkmodell* vor
- 1976: Firma SIEMENS (*UDS*) u.v.a. entwickeln DBS nach dem Netzwerk-Datenmodell
- seit 1970: Codd entwickelt das *Relationale Datenmodell*
- 1971 –1981: IBM entwickelt Prototyp für Relationale Datenbanken, Name: *"System /R"* mit Datenbanksprache: SQL
- seit 1980: *ORACLE*, Relationales DBS mit SQL als Datenbanksprache auf dem Markt
- seit 1982: *SQL/DS* der Firma IBM mit SQL auf dem Markt
- seit 1984: Relationale DBS für PCs auf dem Markt: *DBASE, ACCESS* auch ORACLE
- seit Ende der 80-er: Entwicklung *Objektorientierter DBS*: GEMSTONE, ObjectStore, O2, Jasmine, POET (jetzt: FastObjects) u.v.a.
- 1992: SQL2-Standard wird verkündet; gleichzeitig Arbeit an einer *OO-Erweiterung von SQL*. Arbeitsname: SQL3
- aktuelle Situation: Client-Server–Prinzip beim Einsatz von Datenbanksystemen und Einsatz von Datenbanksystemen in integrierten Softwaresystemen (hauptsächlich noch immer: Relationale DB-Systeme), z.B. in SAP
- Zunehmende Wichtigkeit von Internet-basierten Ansätzen, Big Data

Vorteile von DB-Systemen

- **Redundanzvermeidung -** Ein Fakt in der realen Welt ist nur ein Mal in der DB abgelegt.
- **Vermeidung von Inkonsistenzen -** keine widersprüchlichen Informationen zum gleichen Objekt: Beispiel: Adresse in Datei Personalstamm und in Datei Bestellinfos gespeichert. Person zieht aus Darmstadt aus. Die Datei Personalstamm wird geändert, die Datei Bestellinfos nicht. Durch redundante Datenhaltung wurde Inkonsistenz (Person hat zwei verschiedene Adressen) provoziert.
- **Einheitliche Sicht auf Daten -** Standards setzen sich besser durch
- **Datenunabhängigkeit -** Anwendungen müssen sich nicht um Details der Speicherstruktur und Zugriffstechniken kümmern. Änderungen von Speicherstrukturen, Attributanzahl etc. sind i.d.R. ohne Auswirkungen auf Anwendungsprogramme. Beispiel: Länge des Nachnamens wird von 30 auf 40 Zeichen gesetzt oder zusätzliches Attribut Qualifikation wird eingeführt. Jedoch: Streichen eines Attributs ohne Auswirkungen auf Anwendungsprogramme nicht möglich!
- **Spontane Anfragen -** Mit Hilfe einer Sprache der vierten Generation ist eine Formulierung von Anfragen unter dem Motto „Was“ statt „Wie“ möglich. Beispiel: Selektiere alle Mitarbeiter, die mehr als 50000 € verdienen.

Vorteile von DB-Systemen

- **Zugriffskontrolle bei parallelen Zugriff -** Benutzung von Daten durch mehrere Programme gleichzeitig mit Kontrolle der Zugriffe und eindeutigen Änderungen

- **Integritätskontrolle -** Kontrolle von Integritätsbedingungen durch DBS (z. B. Wertebereiche, Existenzabhängigkeiten zwischen Datensätzen, allg. Beziehungen zwischen Werten), dadurch weniger Programmieraufwand; Beispiele: Alter von Mitarbeitern muss immer > 14 sein. Nachname ist max. 30 Zeichen lang.

- **Datenschutz, Datensicherheit -** Zugriffskontrolle, d. h. Setzen von Sperren durch DBS auch auf Feld- und Satzebene (nicht nur Read-, Write-Sperren ganzer Dateien); Beispiel: Nur Personalabteilung dürfen Gehälter der Mitarbeiter sehen.

- **Beziehungen zwischen Daten -** können definiert und vom System kontrolliert werden. Beispiel: Bestellung hat Beziehung zu Kunde. Zu Bestellung gibt es immer einen Kunden

- **Verteilung von Daten** kann vom DBS verwaltet werden.

DHSN
DUALE HOCHSCHULE
SACHSEN

Historische Entwicklung

Hierarchisches Datenmodell

Netzwerkmodell

Hierarchisches Datenmodell

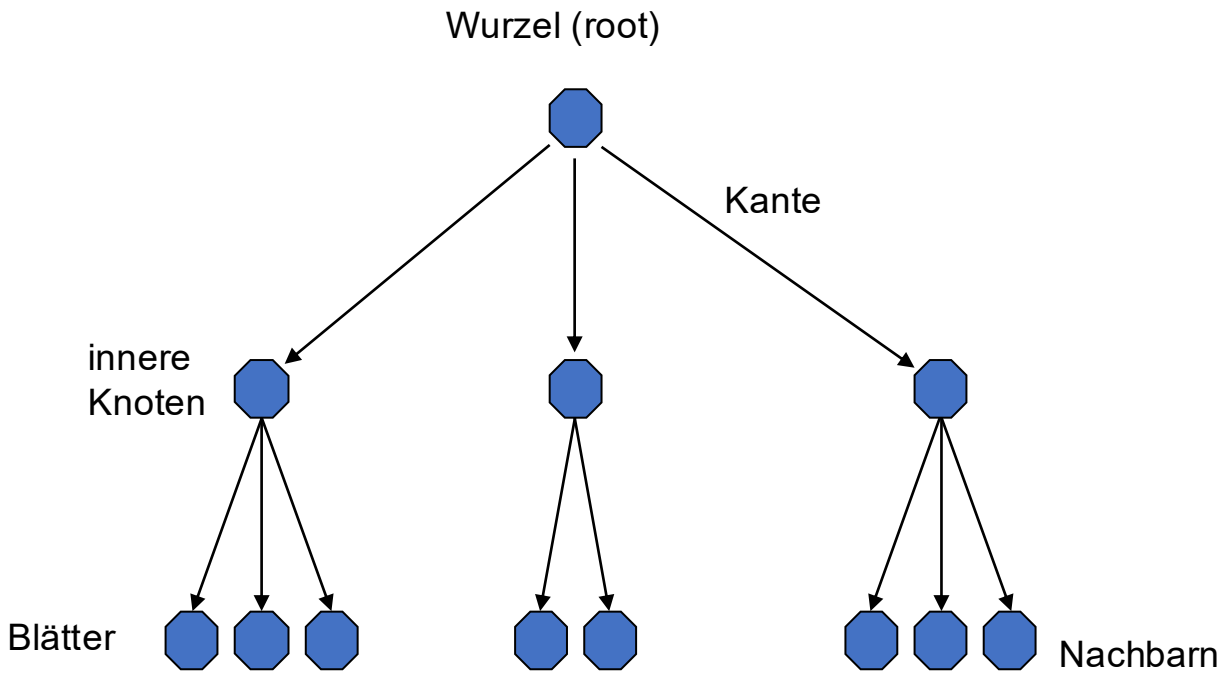
- Ältestes Datenmodell
- Geht aus Informationsmanagementsystemen (IMS) der 50er und 60er Jahre hervor (Einsatzgebiete: Banken und Versicherungsunternehmen)
- Eignet sich für Beziehungen, bei denen sich aus einem Oberbegriff viele Unterbegriffe ableiten lassen (1:n-Beziehungen)
- Zugriff nur über den Suchschlüssel des Objekts der obersten Ebene möglich (die anderen entlang der hierarchischen Ordnung)
- Anwender bzw. Anwendungsprogramm muss den Pfad zum gesuchten Datensatz kennen
- Strukturelemente sind:

  Objekttypen und *unbenannte* hierarchische Beziehungen

  „*unbenannt*“: keine Bezeichnungen für die Beziehung (Gegensatz dazu: Entity-Relationship-Modell)

  Wurzelbaum (Graph aus Objekttypen [Knoten] und Beziehungen [Kanten])

Wurzelbaum



Pfad

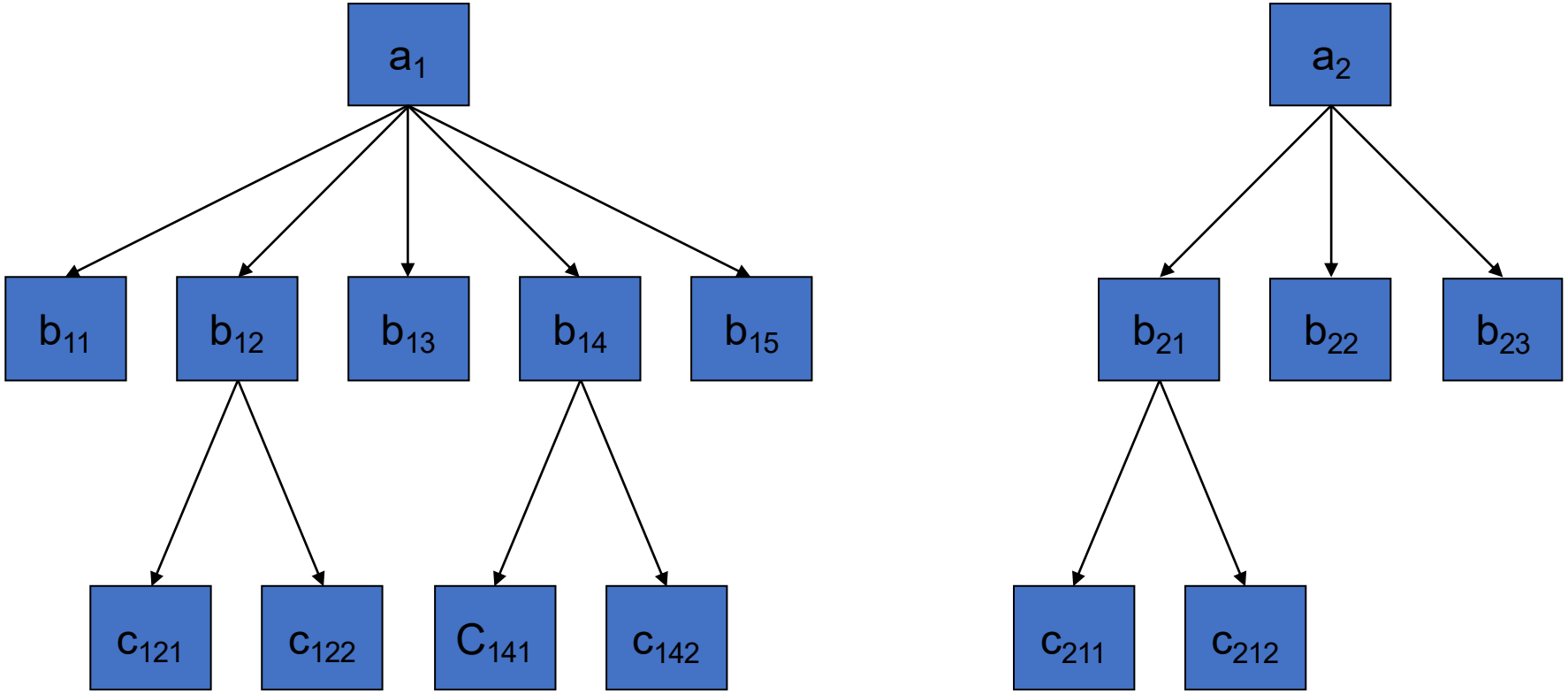
Pfadlänge: hier Länge = 2

Niveau eines Knoten =
Pfadlänge des Knoten von der Wurzel + 1

Höhe eines Wurzelbaums =
max. Pfadlänge

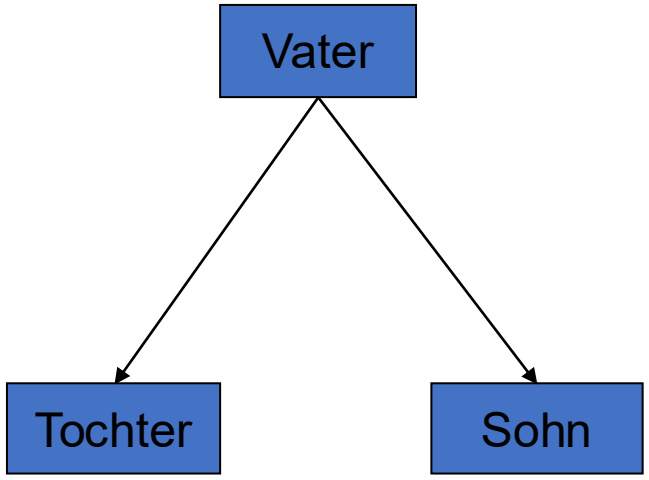
Löhl/Lörcher/Sahm, 2005, S. 13

Hierarchische Ordnung

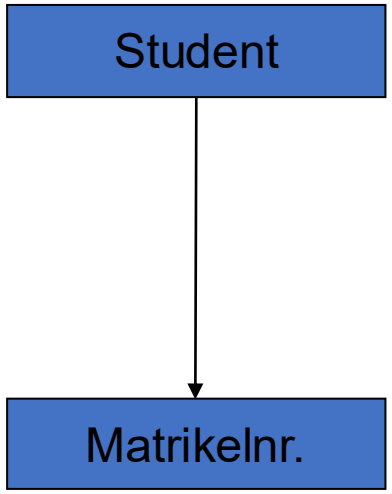


Löhl/Lörcher/Sahm, 2005, S. 14

Darstellung von Beziehungen



1:n - Beziehung



1:1 - Beziehung

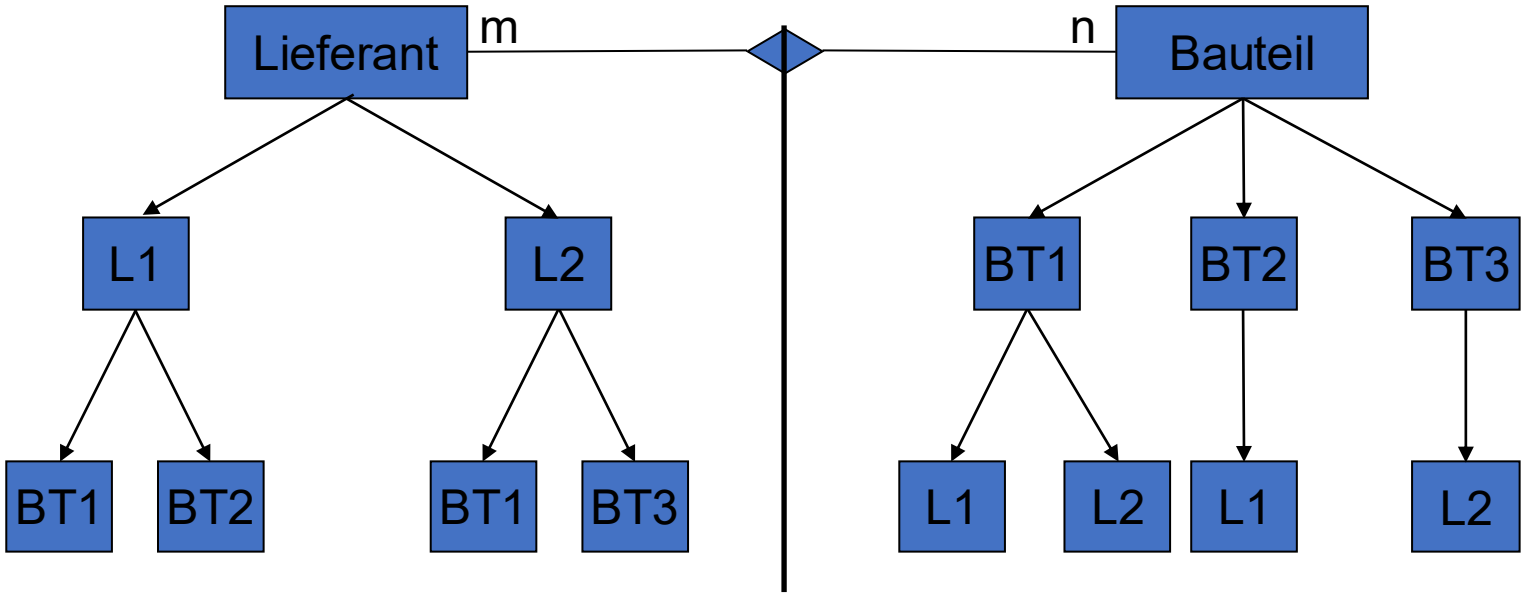
NICHT

MÖGLICH !!

m:n - Beziehung

Löhl/Lörcher/Sahm, 2005, S. 15

Darstellung einer M:N Beziehung

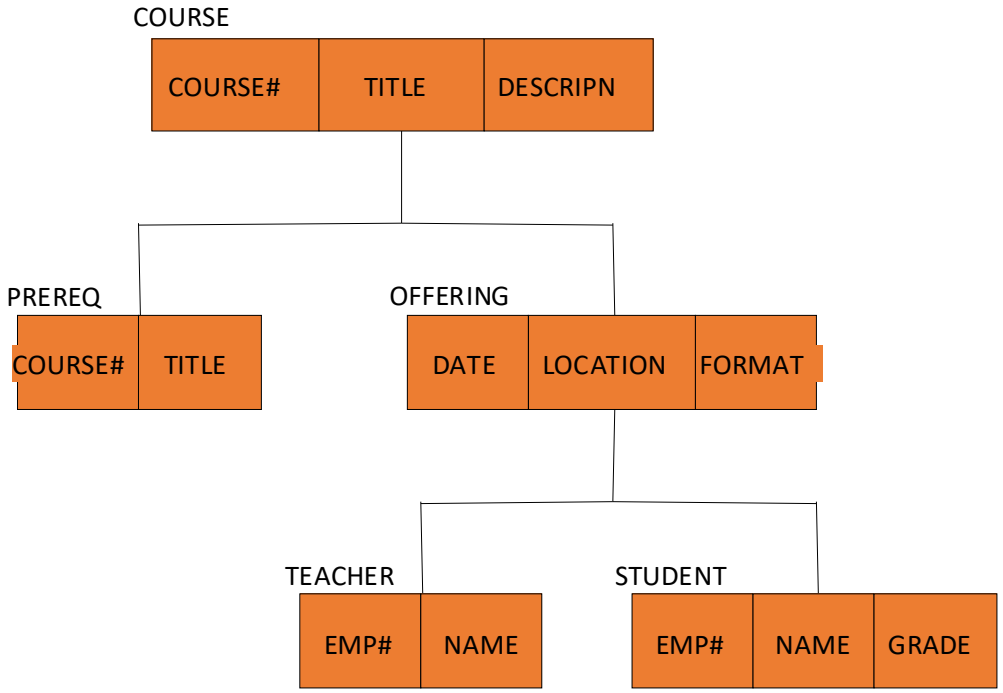


ERHEBLICHE REDUNDANZ !!!!!

Löhl/Lörcher/Sahm, 2005, S. 16

Operation im Hierachischen Modell

Welche Studenten sind im Kurs M23 am 13.08.1973 ?




```
GU COURSE(COURSE#='M23')
   OFFERING (DATE='730813')
if gefunden then
begin
    GNP STUDENT
    while gefunden do
    begin
      write(STUDENT.NAME)
      GNP STUDENT
    end
end;
```

Get Unique (GU) Befehl wird benutzt, um direkt zum spezifischen Segment zu kommen und eine Startposition in der Datenbasis für die nachfolgende sequentielle Durchsuchung zu haben

GNP = Get Next in Parent (GNP) Befehl wird genutzt, um abhängige Segmente sequentiell zu erreichen

Vor- und Nachteile

PRO

- werden Daten in Richtung der Hierarchie gesucht, so ist der Zugriff sehr schnell

CONTRA

- werden Daten gegen die Richtung der Hierarchie gesucht, muss ggf. die gesamt Datenbank durchsucht werden. In diesem Falle sehr langsam, d.h. Abhängigkeit der Performance vom jeweiligen Datenmodell
- Vermischung von interner und externer Ebene
- nur 1:n-Beziehungen
- Zugriff nur durch Anwendungsprogramme

Hierarchische Datenmodelle und darauf basierende DB-Systeme (z.B. das IMS von IBM) werden teilweise noch heute eingesetzt. Einbettung in bestimmte Programmiersprachen (Bsp. COBOL) ist gut gelöst.

DHSN
DUALE HOCHSCHULE
SACHSEN

Historische Entwicklung

Hierarchisches Datenmodell

Netzwerkmodell

Historische Entwicklung – das CODASYL-Konzept (I)

CODASYL (COnference on DAta SYstems Languages)

Gründung im Rahmen eines Treffens von 40 Fachleuten aus Herstellerbranche, Rüstungsindustrie, staatlichen und militärischen Computerzentren am 28. und 29. Mai 1959

Bedarf nach einer maschinen- und herstellerunabhängigen Programmiersprache für Verwaltungsanwendungen

Probleme des Militärs durch Rechnervielfalt und verwaltungstechnische Expansion

Nach sechs Monaten Spezifikation und Veröffentlichung der Programmiersprache COBOL (COmmon Business Oriented Language)

1969: DBTG-Report (DBTG: Data Base Task Group)

1971: Überarbeitete Version des DBTG-Reports → Vorstellung des CODASYL/DBTG-Konzepts

1975: Enhanced Cobol → Cobol erweitert um DB-Manipulationen

CODASYL als solches exisitert heute nicht mehr → einige Ausschüsse arbeiten jedoch noch

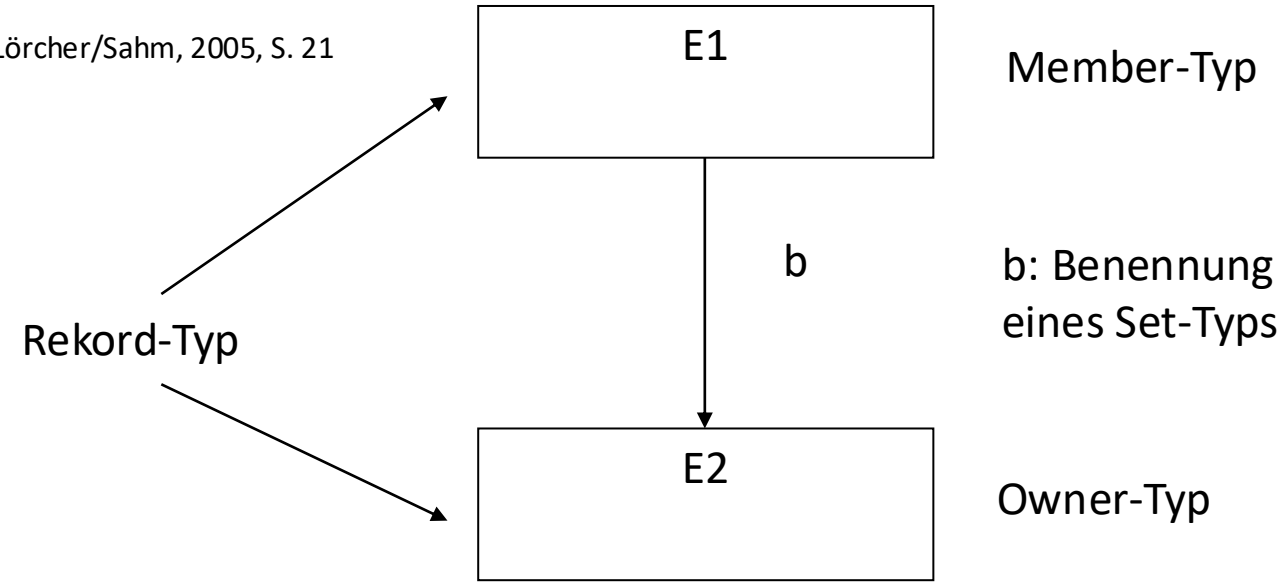
Archivierte Dokumente im Charles Babbage Institute, University of Minnesota

Historische Entwicklung – das CODASYL-Konzept (II)

- 1967 Gründung der Database Task Group (DBTG) Ziel: Entwicklung eines Datenmodells um die Inflexibilität des hierarchischen Datenmodells zu beseitigen
- 1971 hat das COBOL Committee der Database Task Group des CODASYL (COnference on DAta SYstems Language) das Netzwerkdatenmodell als Standard publiziert
- Zunächst nur zwei Ebenen der Datenbeschreibung:
  - Schema (beschreibt die logische Datenstruktur einschließlich der logischen Zugriffspfade)
  - Subschema (beschreibt das Schema aus der Sicht eines Anwendungsprogramms)
- Versuch, die Inflexibilität des Hierarchischen Datenmodells (Vermischung von interner und externer Ebene, nur 1:n-Beziehungen, Abhängigkeit der Performance vom jeweiligen Datenbestand, Zugriff nur durch Anwendungsprogramm) zu beseitigen
- Netzwerkartige Beziehungen lassen sich ohne zusätzliche Konzepte definieren
- Das NDM erfuhr nie die Verbreitung und Akzeptanz, da Codd 1970 sein Relationales Datenmodell veröffentlichte; dieses Modell ging hinsichtlich der Flexibilität und Einfachheit in der Modellierung weit über den CODASYL-Standard hinaus
- Mit der Idee des Semantic Web gewinnt das Netzwerkdatenbankmodell wieder mehr an Bedeutung

Strukturelemente

Löhl/Lörcher/Sahm, 2005, S. 21



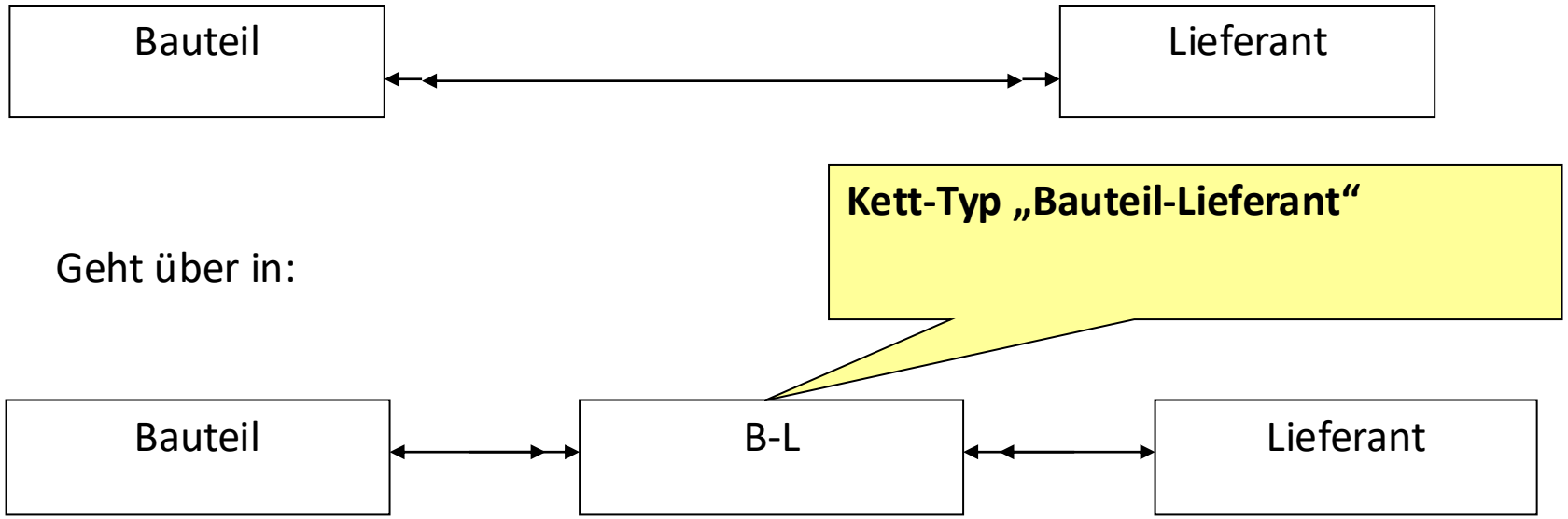
Darstellung von Strukturen

Die Darstellung einer 1:m - Beziehung wird durch einen Set-Typ dargestellt. Die Darstellung von m:n - Beziehungen wird durch einen einfachen Trick ermöglicht:

- Zwischen den beiden Objekttypen wird ein Kett-Objekttyp eingefügt, der mit den beiden anderen Objekttypen je einen Set-Typ darstellt
- Die beiden Objekttypen sind darin jeweils die Owner

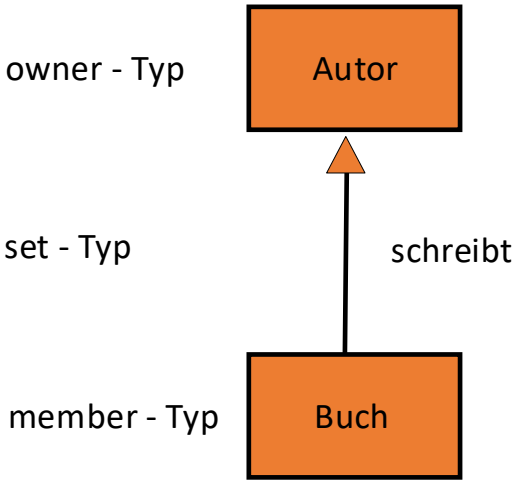
- wie beim Hierarchischen Datenmodell bereits erwähnt: Objekttypen und hierarchische Beziehungen (1:m-Beziehungen), die hier ***Set-Typen*** genannt werden
- im Netzwerkdatenmodell können nur binäre many-one (bzw. one-many)-Beziehungen dargestellt werden
- Set-Typen sind benannte Beziehungen; d.h. sie tragen einen Namen
- ihnen können allerdings keine Attribute zugeordnet werden
- ein Set-Typ ist also eine benannte hierarchische Beziehung ohne Attribute

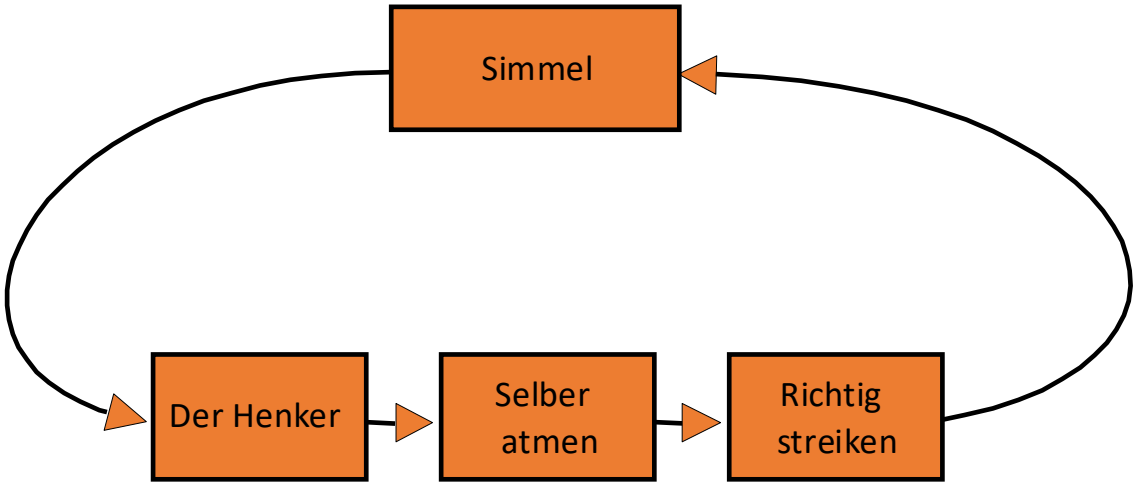
Darstellung von Strukturen im Netzwerkmodell



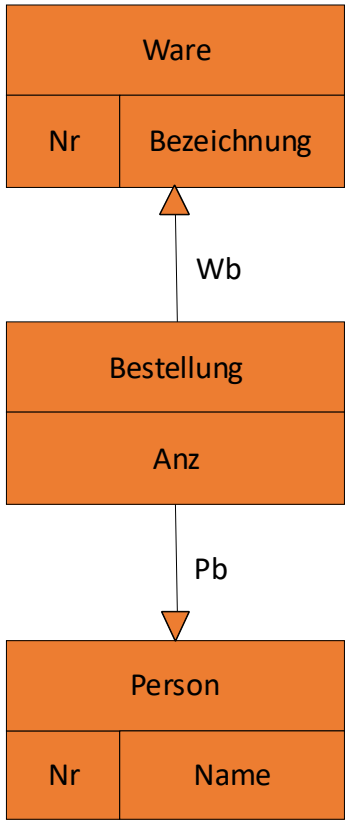
Löhl/Lörcher/Sahm, 2005, S. 24

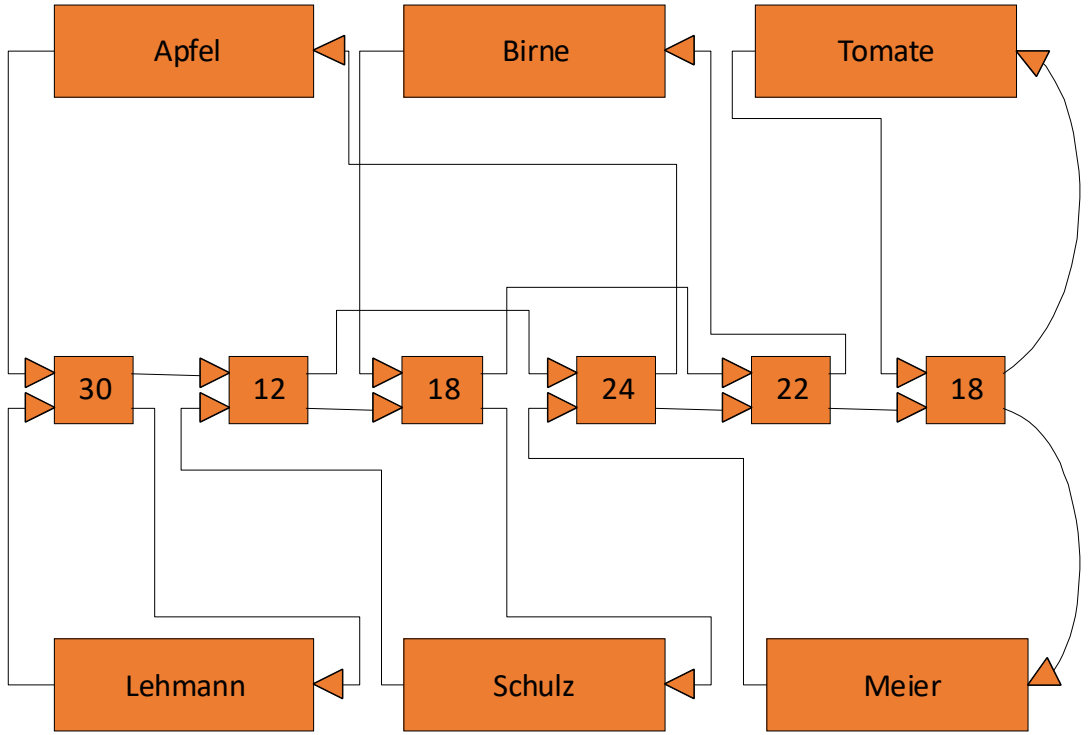
Darstellung von Strukturen





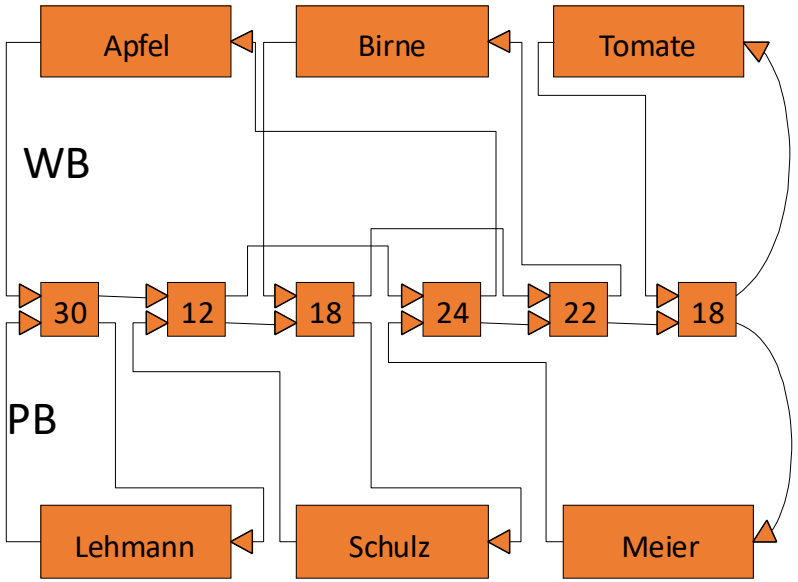
Kettrecords mit Attribut im Netzwerkmodell





Operationen im Netzwerkmodell

Typische Operation : Navigation durch die verzeigerten Entities




```
PERSON.NAME   := 'SCHULZ';
FIND ANY PERSON USING NAME;
IF GEFUNDEN THEN
BEGIN
    FIND FIRST BESTELLUNG WITHIN PB;
    WHILE GEFUNDEN DO
    BEGIN
        FIND OWNER WITHIN WB;
        GET WARE;
        WRITE(WARE.BEZEICHNUNG);
        FIND NEXT BESTELLUNG WITHIN PB;
    END
END;
```

Vor- und Nachteile

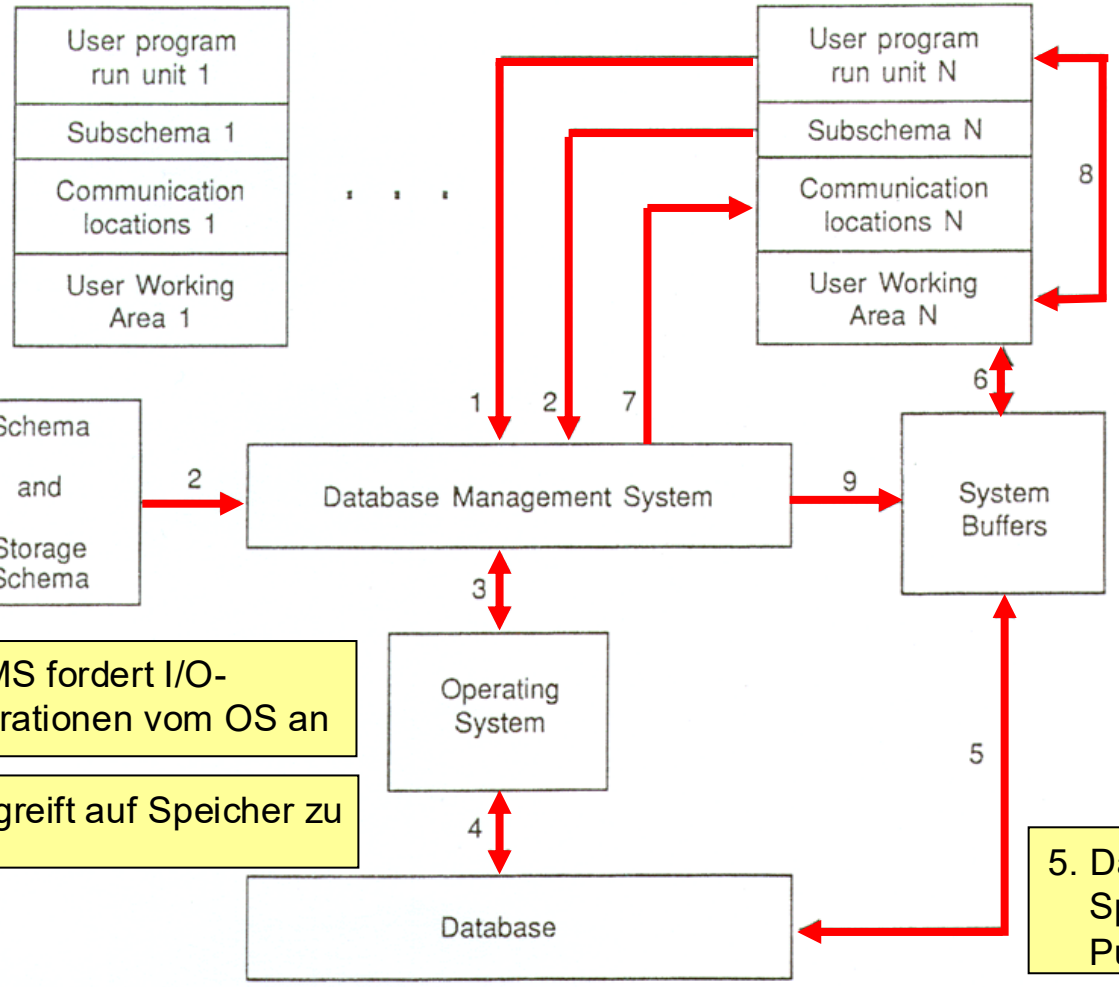
Stärken

- Normierter Zugriff und Modellierung.
- Sehr effizient, wenn die Verarbeitung der physischen Organisation entspricht
- Keine strenge Hierarchie durch Abbildung von m:n-Beziehungen

Schwächen

- Mangelhafte Datenunabhängigkeit:, Kleine Änderungen an der Datenorganisation haben gewaltige Auswirkungen auf die Programme.
- Komplexe Modellierung, da nur eingeschränkte Mechanismen verfügbar.
- Anwendungsprogramme sind sehr komplex und schwer wartbar

Das CODASYL-Konzept



1. Anforderung der Daten vom DBMS

2. DBMS wertet Anforderung, Subschema und Schema aus

3. DBMS fordert I/O-Operationen vom OS an

4. OS greift auf Speicher zu

5. Datentransfers zwischen Speichern und System-Puffern

6. Datentransfer zwischen System-Puffern und Arbeitsbereich der Anwendung entsprechend Subschema und Anforderung

7. DBMS stellt dem Anwenderprogramm Statusinformationen zur Verfügung (z. B. Fehlermeld.)

8. Anwenderprogramm verarbeitet die Daten

9. DBMS verwaltet System-Puffer für verschiedene Anwenderprogramme

Das CODASYL-Konzept

CODASYL/DBTG-Konzept wurde nach seiner Veröffentlichung als Verbesserung gegenüber dem hierarchischen Datenmodell begrüßt

Führende Datenbankhersteller boten darauf basierende DBMS an:

- Siemens: UDS
- Unisys: DMS-1100
- Honeywell: IDS
- DEC: DBMS 10

Allerdings erreichte das CODASYL/DBTG-Konzept nie die vorhergesagte Akzeptanz und Verbreitung

Gründe:

- keine Unterstützung durch IBM (Verhinderung der Entstehung eines Standards)
- Veröffentlichung des relationalen Datenbankmodells durch Ted Codd 1970

→Modellierung einfacher und flexibler

→noch heute gebräuchlich

Das war‘s für heute

- Löhl/Lörcher/Sahm, 2005, Fachhochschule Heilbronn, Vortrag Historische Entwicklung von Datenbanken
- KEMPER, A.; EICKLER, A.: Datenbanksysteme: Eine Einführung, Oldenbourg: München, 10. Auflage (2015)
- ELMASRI, R.; NAVATHE, S. B.: Grundlagen von Datenbanksystemen: Bachelorausgabe Pearson Studium: München 2009
- Laudon, Kenneth C.; Laudon, Jane P.; Schoder, Detlef: Wirtschaftsinformatik, Eine Einführung (3. Auflage, 2015)